DFSha: Sistema de Archivos Distribuido por Bloques con Alta Disponibilidad sobre AWS

Proyecto 1: Sistemas Distribuidos

Universidad EAFIT, 2026-2

Autor	Matías Martínez Moreno
Profesor	Álvaro Ospina Sanjuan
Opción asignada	Opción 1: cliente/servidor con servicio distribuido por composición (S2S)
Repositorio	https://github.com/matias-martinez-moreno/DFSha-Sistema-de-Archivos-Distribuido-por-Bloques-con-Alta-Disponibilidad-sobre-AWS
Video	https://youtu.be/EXfGDpnk5QI
Fecha	15 de Octubre de 2026

Contenido

1. Objetivo 4
Objetivo general 4
Objetivos específicos 4
2. Marco teórico 4
3. Descripción del servicio 5
3.1 Problema 5
3.2 Qué es DFSha 5
3.3 Requisitos funcionales 5
3.4 Requisitos no funcionales 6
3.5 Alcance 6
4. Arquitectura 6
4.1 Tipo de arquitectura 6
4.2 Componentes 7
4.3 Qué guarda cada nodo 8
4.4 Despliegue 9
5. Protocolos y APIs 10
5.1 Protocolo 10
5.2 API del ControlNode (puerto 8000) 10
5.3 API del DataNode (puerto 8100) 11
5.4 Escritura de un archivo (put) 12
5.5 Lectura de un archivo (get) 13
5.6 Heartbeats y órdenes a los DataNodes 14
6. Particionamiento 14
6.1 Partición en bloques 14
6.2 Ubicación de bloques: round robin 14
6.3 Reparto de las lecturas 15
7. Replicación y consistencia 15
7.1 Replicación de los bloques 15
7.2 Replicación de los metadatos 16
7.3 Modelo de consistencia 17

7.4 DFSha frente al teorema CAP 17
8. Alta disponibilidad 18
8.1 Re-replicación 18
8.2 Cambio de líder 19
9. Escalabilidad y rendimiento 20
9.1 Decisiones 20
9.2 Límites 20
9.3 Ley de Little 20
10. Seguridad 21
11. Transparencia 22
12. Entorno de ejecución 22
12.1 Tecnologías 22
12.2 Tres formas de ejecutarlo 23
12.3 Despliegue en AWS 23
13. Pruebas y resultados 24
13.1 Plan 24
13.2 Pruebas automáticas (P1 a P7) 24
13.3 P2: distribución de bloques en AWS 26
13.4 P3: caída de un DataNode en AWS 26
13.5 P4: caída del ControlNode líder en AWS 27
13.6 P5 y P6: consistencia en AWS 27
13.7 P7: seguridad en AWS 27
13.8 P8: carga con Locust 28
13.9 P9: escalabilidad 28
13.10 P10: transferencia según tamaño y número de hilos 29
13.11 Observaciones durante las pruebas 29
14. Limitaciones y trabajo futuro 30
15. Conclusiones 30

1. Objetivo

Objetivo general

Diseñar e implementar DFSha, un sistema de archivos distribuido que guarda archivos grandes repartidos en bloques entre varios nodos, sigue funcionando cuando un nodo falla, protege los datos y le oculta al usuario dónde está cada parte.

Objetivos específicos

1. Construir un cliente con CLI y SDK en Python que permita gestionar el árbol de archivos, subir y bajar archivos, y leer o escribir con locks.
2. Partir cada archivo en bloques y repartirlos entre los DataNodes con round robin, guardando 3 copias de cada bloque.
3. Mantener los metadatos en dos ControlNodes (líder y seguidor) con cambio automático de líder, y definir un modelo de consistencia claro.
4. Proteger el sistema con TLS, cifrado en disco, usuarios y permisos, desplegarlo en AWS con Terraform y Docker, y medir su comportamiento.

2. Marco teórico

Sistemas de archivos distribuidos. Permiten acceder a archivos guardados en varias máquinas como si estuvieran en una sola. NFS es el más conocido: monta una carpeta remota y el sistema operativo la trata como local.

GFS y HDFS. Un nodo maestro (Master o NameNode) guarda solo metadatos: el árbol de directorios, los bloques de cada archivo y dónde están. Los trabajadores (DataNodes) guardan los bloques. El cliente le pregunta al maestro dónde ir y luego lee o escribe directo en los trabajadores, así el maestro no carga con los datos. Los trabajadores le avisan al maestro que siguen vivos con un heartbeat.

Particionamiento. Consiste en dividir los datos en partes para repartir datos y carga entre nodos. Las estrategias del particionamiento horizontal son por rango de llaves, round robin, aleatorio y por hash. El rango puede crear hot spots y el hash complica las consultas por rango. Round robin asigna en orden circular y reparte muy parejo.

Replicación. Tener copias de un dato en varios nodos da tolerancia a fallas y mejores lecturas, pero obliga a mantener las copias iguales. Con un solo líder, todas las escrituras pasan por él y las propaga a los seguidores en el mismo orden; la replicación puede ser síncrona (el líder espera la confirmación) o asíncrona. Si el líder falla, un seguidor toma su lugar. Con varios líderes aparecen conflictos que hay que resolver (última escritura gana, relojes vectoriales, quórum).

CAP y consistencia. Cuando la red falla, un sistema distribuido no puede ser a la vez consistente y disponible, hay que escoger. Los modelos de consistencia dicen qué ve cada lector: en la estricta

siempre la última escritura, en la secuencial un mismo orden global de operaciones, en la eventual las copias terminan iguales con el tiempo.

Escalabilidad. Es la capacidad de atender más carga sin perder eficiencia, ya sea con una máquina más grande (vertical) o con más máquinas (horizontal). Las técnicas básicas son particionar, replicar, comunicarse en paralelo y de forma asíncrona. La ley de Little ($L = \lambda \cdot W$) relaciona las peticiones dentro del sistema (L) con la tasa de llegada (λ) y el tiempo de servicio (W).

3. Descripción del servicio

3.1 Problema

Un solo servidor de archivos se cae y deja sin servicio, pierde los datos si se daña su disco y tiene un tope de capacidad y de velocidad. DFSha reparte cada archivo en bloques entre varios servidores, guarda varias copias de cada bloque y separa los metadatos de los datos.

3.2 Qué es DFSha

Es un sistema con arquitectura tipo HDFS, formado por tres piezas:

1. Dos **ControlNodes**, uno líder y uno seguidor, que guardan los metadatos: árbol de directorios, bloques de cada archivo, usuarios y permisos.
2. Varios **DataNodes** (cuatro en el despliegue) que guardan los bloques cifrados.
3. Un **cliente** con CLI y SDK en Python.

El servicio lo diseña y opera una sola organización, ofrece una API abierta al cliente (REST sobre HTTPS), mantiene la comunicación entre servidores como algo interno y corre dentro de una red privada virtual (VPC de AWS).

3.3 Requisitos funcionales

Id	Requisito	Cómo se ofrece
RF1	Gestión del sistema de archivos	`ls`, `cd`, `pwd`, `mkdir`, `rmdir`, `rm`, `chmod`, `chgrp`, `info`
RF2	Transferencia de archivos	`put` y `get`: mueven el archivo completo, partido en bloques
RF3	Acceso a archivos	`open` en lectura o escritura, `read` por rango de bytes, `write`, `close`, `lock`, `unlock`
RF4	Usuarios	`login`, `whoami`, `useradd` (solo admin); cada usuario tiene su carpeta `/users/<usuario>`
RF5	Estado del clúster	`nodes` muestra ControlNodes, DataNodes y bloques con pocas réplicas

La diferencia entre transferencia y acceso es que `put` y `get` mueven el archivo completo, mientras que con `open` y `read` el cliente lee una parte y el sistema baja solo los bloques que la cubren.

3.4 Requisitos no funcionales

Requisito	Cómo se cumple
Particionamiento	Bloques de 4 MB repartidos por round robin
Alta disponibilidad	3 réplicas por bloque, re-replicación automática, ControlNode líder y seguidor
Consistencia	Log de operaciones replicado, commit atómico, locks
Rendimiento y concurrencia	El ControlNode no mueve datos y los bloques viajan en paralelo
Escalabilidad	DataNodes que se suman con Terraform y se registran solos
Seguridad	TLS, AES-256-GCM en disco, bcrypt, JWT, permisos, llave interna
Transparencia	Rutas lógicas y ubicación consultada en cada operación

3.5 Alcance

1. Los archivos se escriben completos (WORM): modificar un archivo es subir una versión nueva que reemplaza a la anterior. No hay escritura parcial.
2. Solo el ControlNode líder atiende operaciones; el seguidor está listo para reemplazarlo.
3. Los valores por defecto son bloques de 4 MB, 3 réplicas por bloque y mínimo 2 confirmadas para aceptar una escritura.

4. Arquitectura

4.1 Tipo de arquitectura

Es cliente/servidor, y el servidor tiene un esquema de maestro y trabajadores: los ControlNodes son los maestros de metadatos y los DataNodes los trabajadores. Se separó así porque los metadatos son pocos y necesitan orden, mientras que los datos son grandes y necesitan escalar.

4.2 Componentes

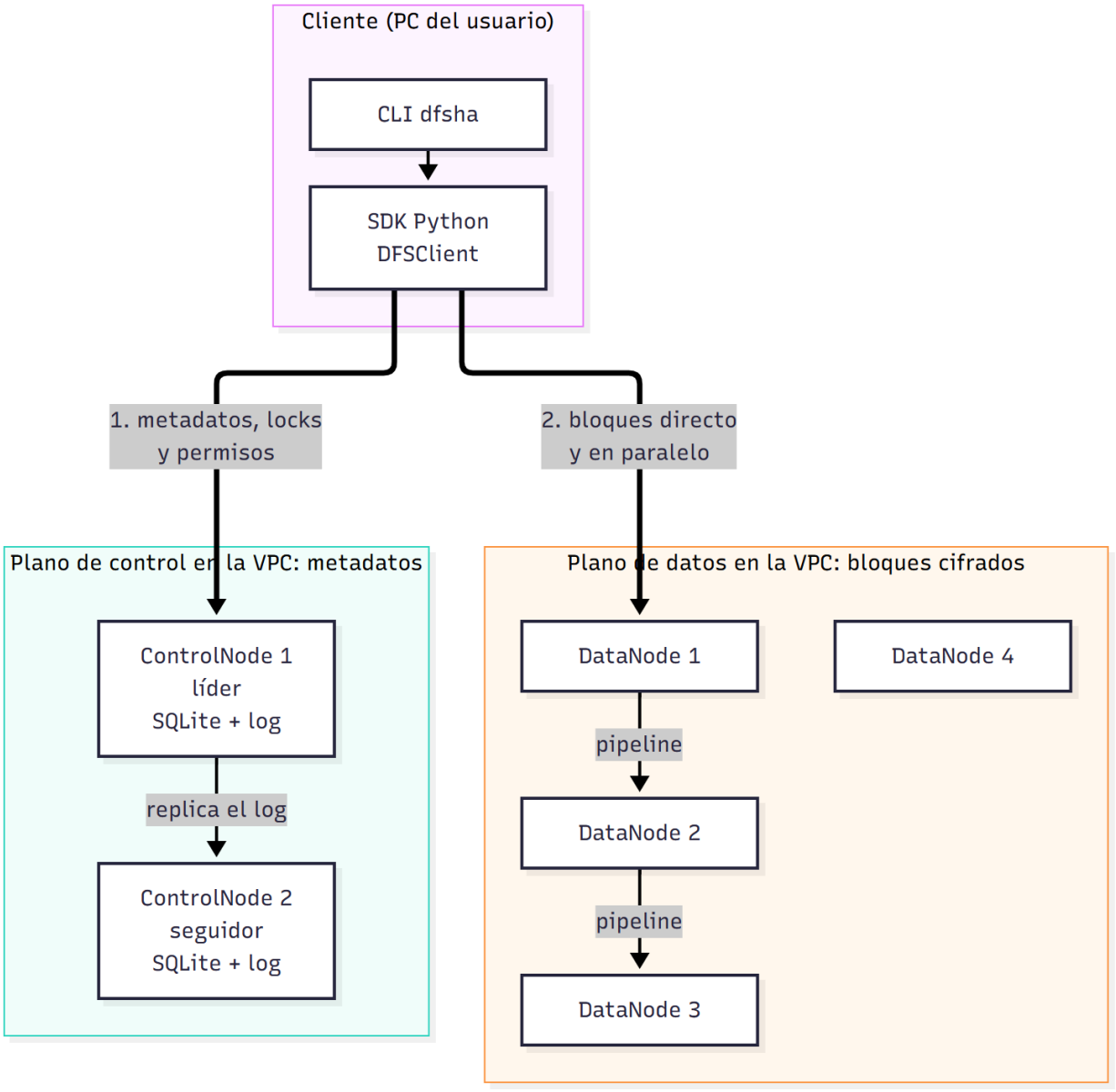


Figura 1. Componentes

Componente	Responsabilidad	Tecnología
Cliente	Partir y armar archivos, subir y bajar bloques en paralelo, verificar checksums, encontrar al líder	Python, httpx
ControlNode	Metadatos, usuarios y permisos, locks, plan de ubicación, monitor de réplicas, replicación del log	Python, FastAPI, SQLite
DataNode	Guardar bloques cifrados, copiarlos en cadena, re-replicar por orden del líder	Python, FastAPI, AES-GCM

4.3 Qué guarda cada nodo

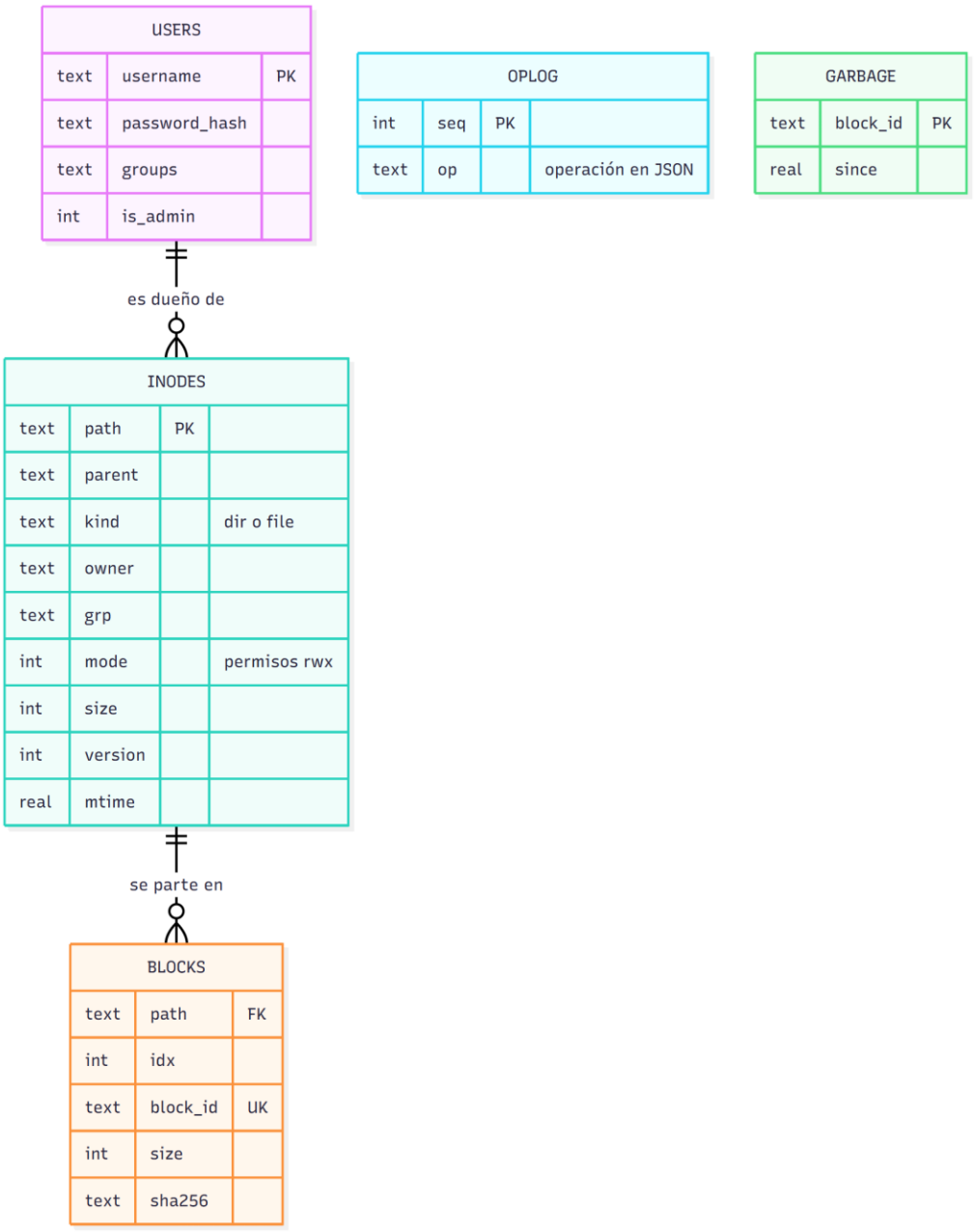


Figura 2. Qué guarda cada nodo

Los ControlNodes guardan estas tablas en SQLite. `oplog` es el log de operaciones: cada cambio se guarda con un número de secuencia y es lo que se replica al seguidor. `garbage` tiene los bloques que ya no pertenecen a ningún archivo y que hay que borrar de los DataNodes.

En qué DataNode está cada bloque no se guarda en la base: se arma en memoria con los reportes de heartbeat, como en HDFS. Así, si un DataNode cae o vuelve, el mapa se corrige solo. Los DataNodes guardan cada bloque como un archivo `<id>.blk` cifrado.

4.4 Despliegue

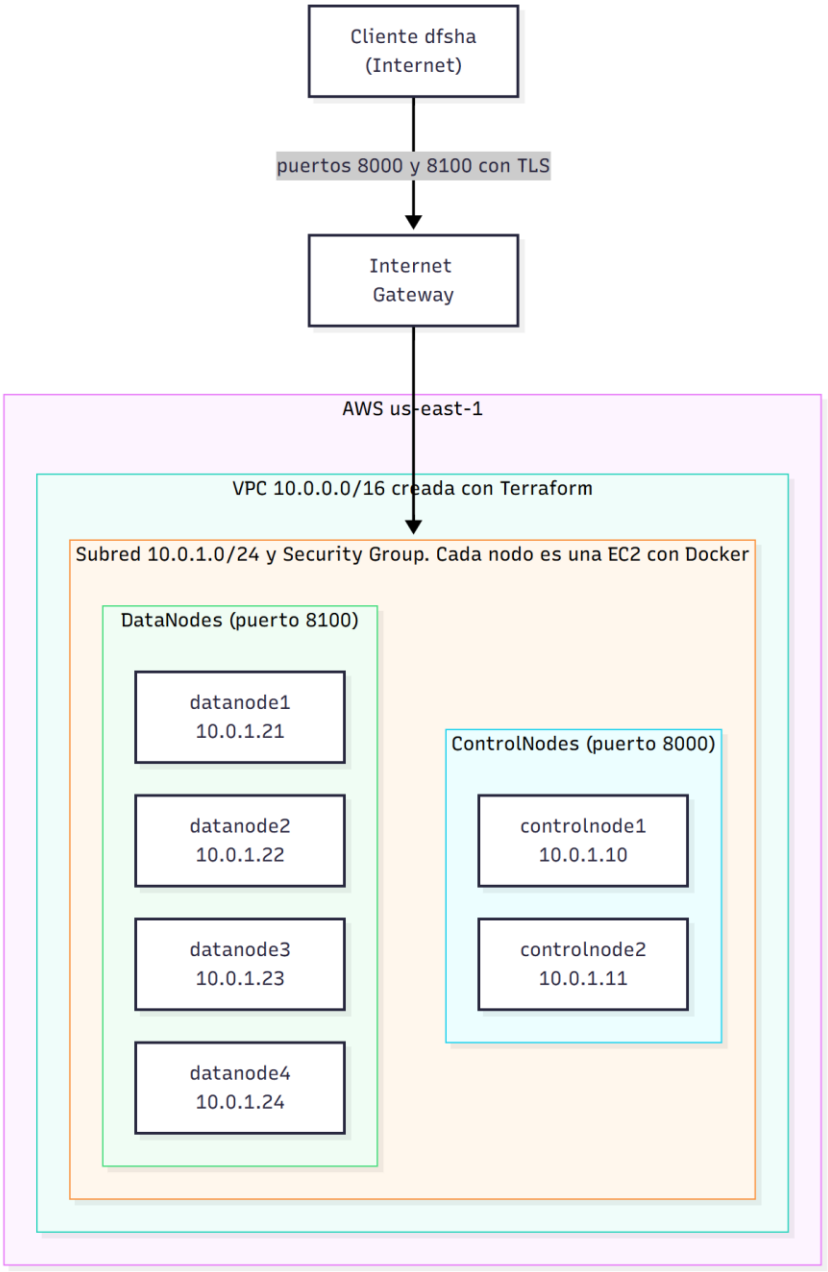


Figura 3. Despliegue

Cada nodo corre en su propia EC2 dentro de un contenedor Docker, con IP privada fija. El Security Group abre a Internet los puertos 8000 (ControlNode) y 8100 (DataNode), que son los que usa el cliente, y el 22 para administrar las máquinas por SSH, que se puede limitar a una sola IP con allowed_ssh_cidr.

5. Protocolos y APIs

5.1 Protocolo

Toda la comunicación es REST sobre HTTPS con JSON; los bloques viajan como binario dentro del cuerpo HTTP. Se escogió REST porque es fácil de depurar.Lo único asíncrono son las órdenes del líder a los DataNodes (borrar o re-replicar), y van en la respuesta al heartbeat, como en HDFS, así que no hizo falta un broker de mensajes.

Par	Para qué	Endpoints	Autenticación
Cliente a ControlNode	Login, metadatos, locks, planes de lectura y escritura	`/auth/*`, `/fs/*`, `/locks`, `/files/*`, `/cluster`	JWT del usuario
Cliente a DataNode	Subir y bajar bloques	`PUT` y `GET /blocks/{id}`	Permiso de bloque firmado por el ControlNode
ControlNode a ControlNode	Replicar el log, copiar el log completo, consultar estado	`/internal/replicate`, `/internal/log`, `/status`	Llave interna y red privada
DataNode a ControlNode	Registro, heartbeat y reporte de bloques	`/internal/heartbeat`	Llave interna y red privada
DataNode a DataNode	Copia en cadena y re-replicación	`PUT /internal/blocks/{id}`	Llave interna y red privada

5.2 API del ControlNode (puerto 8000)

Método y ruta	Qué hace	Quién
`POST /auth/login`	Devuelve el JWT	Cualquiera
`GET /auth/me`	Datos del usuario y su carpeta	Usuario
`POST /users`	Crea un usuario con su carpeta	Admin
`GET /fs/ls`, `GET /fs/stat`	Lista una carpeta o describe una entrada	`r` sobre la ruta
`POST /fs/mkdir`, `DELETE /fs/rmdir`, `DELETE /fs/rm`	Crear carpeta, borrar carpeta vacía, borrar archivo	`w` en la carpeta padre
`POST /fs/chmod`, `POST /fs/chgrp`	Cambiar permisos o grupo	Dueño
`GET /fs/blocks`	Bloques del archivo y nodos que los tienen	`r`
`POST /locks`, `DELETE /locks/{id}`	Tomar o soltar un lock de lectura o escritura	`r` o `w`
`POST /files/plan-write`	Número de bloques, destinos y permiso firmado por bloque	Lock `w` vigente
`POST /files/commit`	Publica el archivo cuando todos los bloques están escritos	Lock `w` vigente
`GET /files/plan-read`	Bloques con checksum, réplicas vivas y permiso	Lock `r` vigente

Método y ruta	Qué hace	Quién
`GET /cluster`, `GET /status`	Estado del clúster y del nodo	Usuario / público

Si una petición llega al seguidor, este responde `503 not_leader` y el SDK busca al líder y reintenta.

5.3 API del DataNode (puerto 8100)

Método y ruta	Encabezados	Respuesta
`PUT /blocks/{id}`	Permiso de bloque, `X-Block-SHA256`, `X-DFS-Pipeline` con las URLs de las otras réplicas	Nodos donde quedó guardado
`GET /blocks/{id}`	Permiso de bloque	Bytes del bloque
`PUT /internal/blocks/{id}`	Llave interna, `X-Block-SHA256`, `X-DFS-Pipeline`	Nodos donde quedó guardado
`GET /status`		Nodo, cantidad de bloques y espacio usado

5.4 Escritura de un archivo (put)

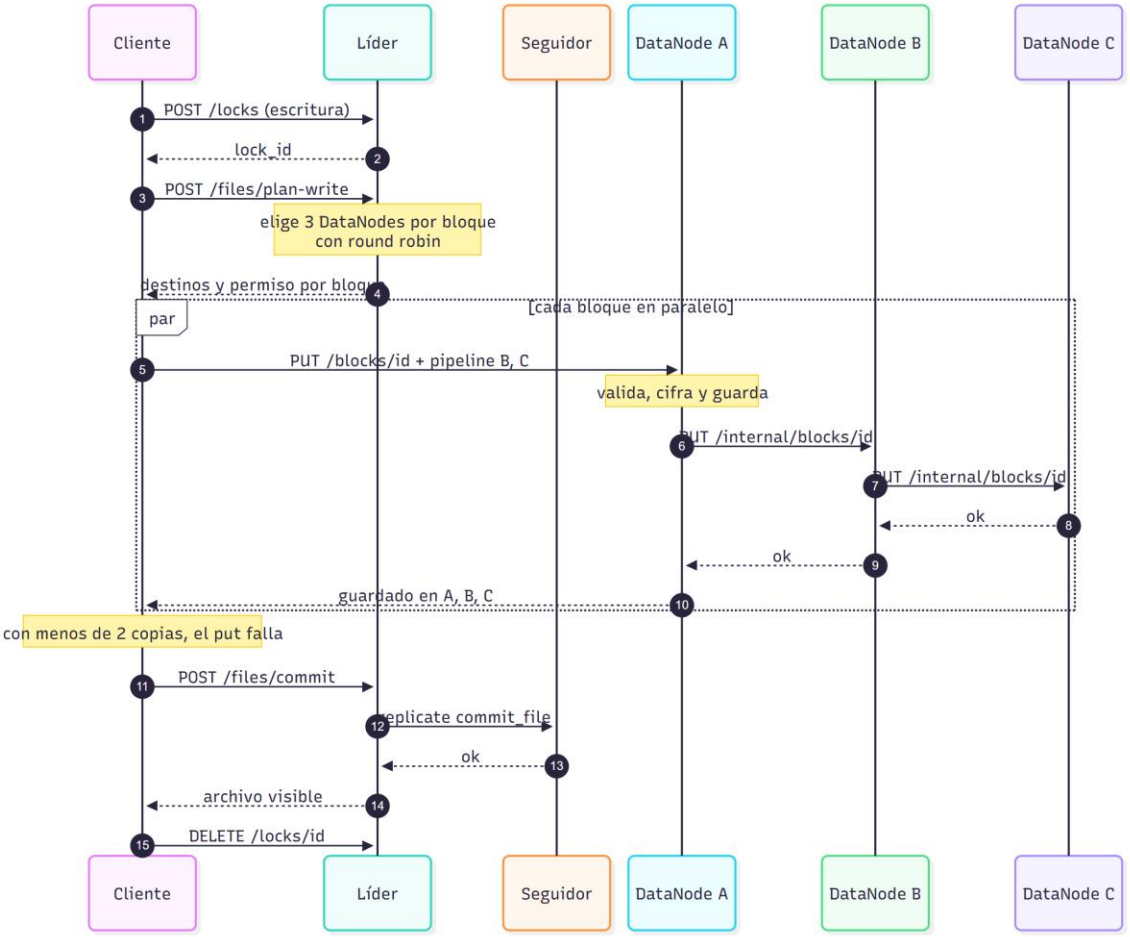


Figura 4. Escritura de un archivo (put)

5.5 Lectura de un archivo (get)

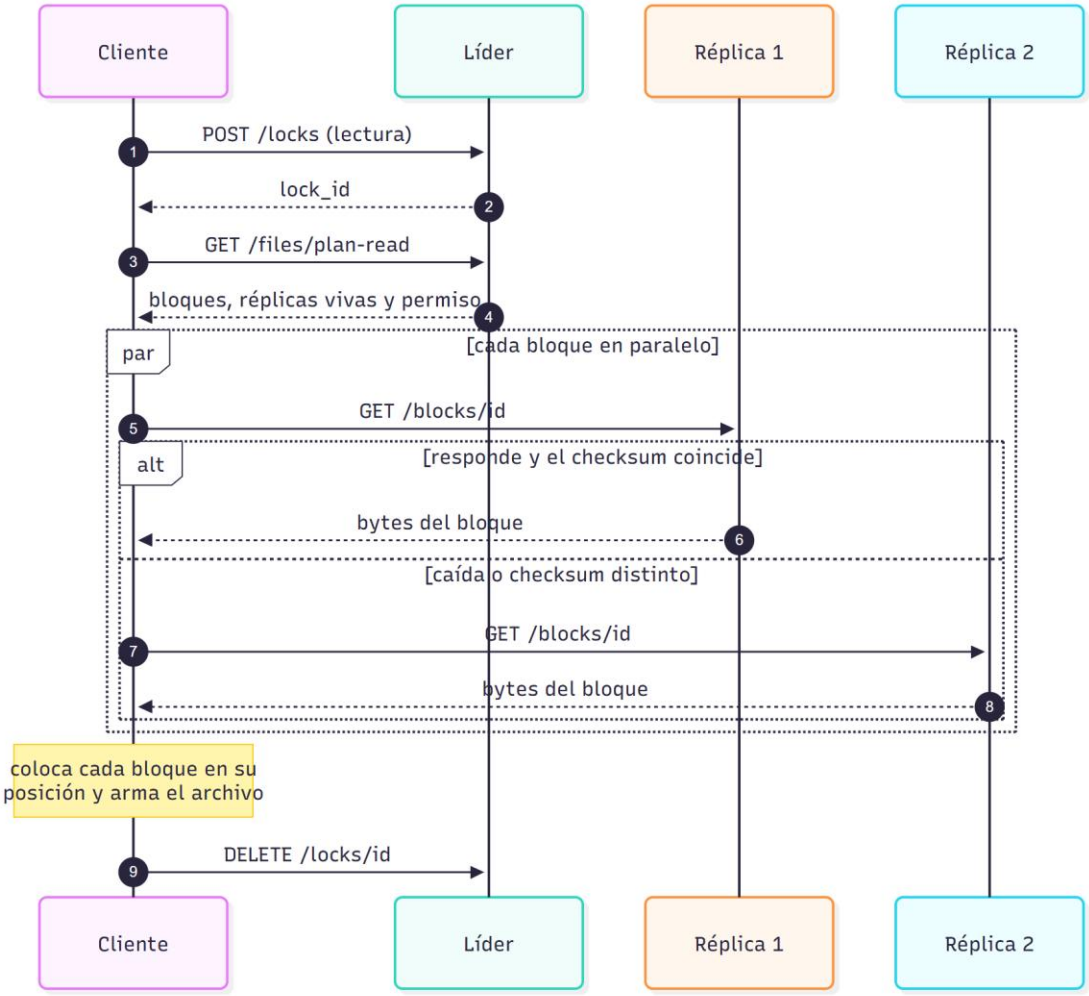


Figura 5. Lectura de un archivo (get)

5.6 Heartbeats y órdenes a los DataNodes

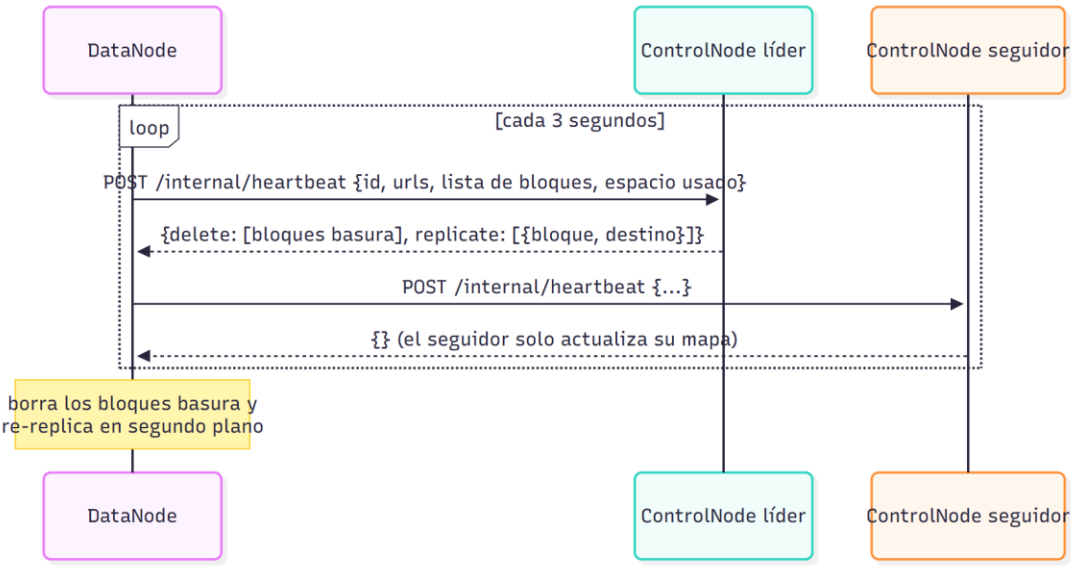


Figura 6. Heartbeats y órdenes a los DataNodes

6. Particionamiento

6.1 Partición en bloques

El cliente parte cada archivo en bloques de 4 MB; un archivo de tamaño S queda en `ceil(S / 4 MB)` bloques y el último puede ser más pequeño. Cada bloque recibe un id aleatorio de 128 bits que no depende del nombre del archivo, así sobrescribir o renombrar no obliga a mover bloques.

Se escogió 4 MB como punto medio para este proyecto. HDFS usa 128 MB, pero con archivos de decenas de MB quedarían en un solo nodo y no se vería el reparto. Es configurable con `BLOCK_SIZE`.

Archivo de prueba	Tamaño	Bloques
libro.pdf	42,7 MB	11
vide190mb.mp4	190,1 MB	48

6.2 Ubicación de bloques: round robin

El líder tiene un contador global. Para cada bloque nuevo toma de la lista de DataNodes vivos el que está en la posición `contador mod N` como primera réplica, las otras dos van en los nodos que siguen, y el contador avanza.

```
vivos = DataNodes con heartbeat en los últimos 10 s, ordenados
copias = min(factor_de_replicación, len(vivos))
para cada bloque:
    inicio = contador mod len(vivos)
    contador = contador + 1
    destinos = [vivos[(inicio + j) mod len(vivos)] para j en 0..copias-1]
```

Ejemplo con 4 DataNodes y factor 3, tomado de los primeros bloques de `libro.pdf` en AWS:

Bloque	Réplica 1	Réplica 2	Réplica 3
0	datanode1	datanode2	datanode3
1	datanode2	datanode3	datanode4
2	datanode1	datanode3	datanode4
3	datanode1	datanode2	datanode4

Así ningún nodo tiene el archivo completo y todos tienen una parte, que es lo que pide el particionamiento. Se prefirió round robin sobre rango de llaves (puede formar hot spots) y sobre hash (también reparte bien, pero con pocos nodos round robin queda más parejo y es fácil de verificar). Su desventaja, no saber dónde quedó un dato sin un índice, no aplica porque el ControlNode siempre sabe dónde está cada bloque.

6.3 Reparto de las lecturas

Al leer, el ControlNode devuelve las réplicas vivas de cada bloque en orden aleatorio y el cliente prueba primero la primera. Así las lecturas de un mismo archivo se reparten entre sus tres copias.

7. Replicación y consistencia

7.1 Replicación de los bloques

1. **Factor 3:** cada bloque queda en tres DataNodes distintos.
2. **Copia en cadena:** el cliente manda el bloque solo al primer DataNode, junto con la lista de los otros dos. Ese lo guarda y lo reenvía al segundo, y el segundo al tercero. El cliente lo sube una sola vez y las copias viajan por la red privada.
3. **Mínimo de 2:** la escritura de un bloque se acepta si al menos dos copias confirmaron. Si un nodo de la cadena está caído, el anterior salta al siguiente. Con menos de dos, el `put` falla y el archivo no se publica.
4. **Re-replicación:** si un bloque queda con menos de tres copias vivas, el líder lo detecta y lo copia a otro nodo (sección 8).

Los bloques nunca se modifican. Una versión nueva de un archivo usa bloques nuevos y los viejos se borran después, así que entre las copias de un bloque no hay conflictos: o está completo e igual (lo garantiza el checksum SHA-256) o no está.

7.2 Replicación de los metadatos

Los metadatos usan un solo líder con replicación síncrona:

1. Solo el líder acepta operaciones de metadatos.
2. Cada cambio se convierte en una operación con número de secuencia y se guarda en el log.
3. El líder aplica la operación y se la envía al seguidor, y espera su confirmación antes de responderle al cliente. El seguidor la aplica en el mismo orden.
4. Si al seguidor le falta alguna operación, la pide al líder (copia incremental).
5. Un ControlNode que arranca o vuelve de una falla borra su copia local y repite el log completo del líder.

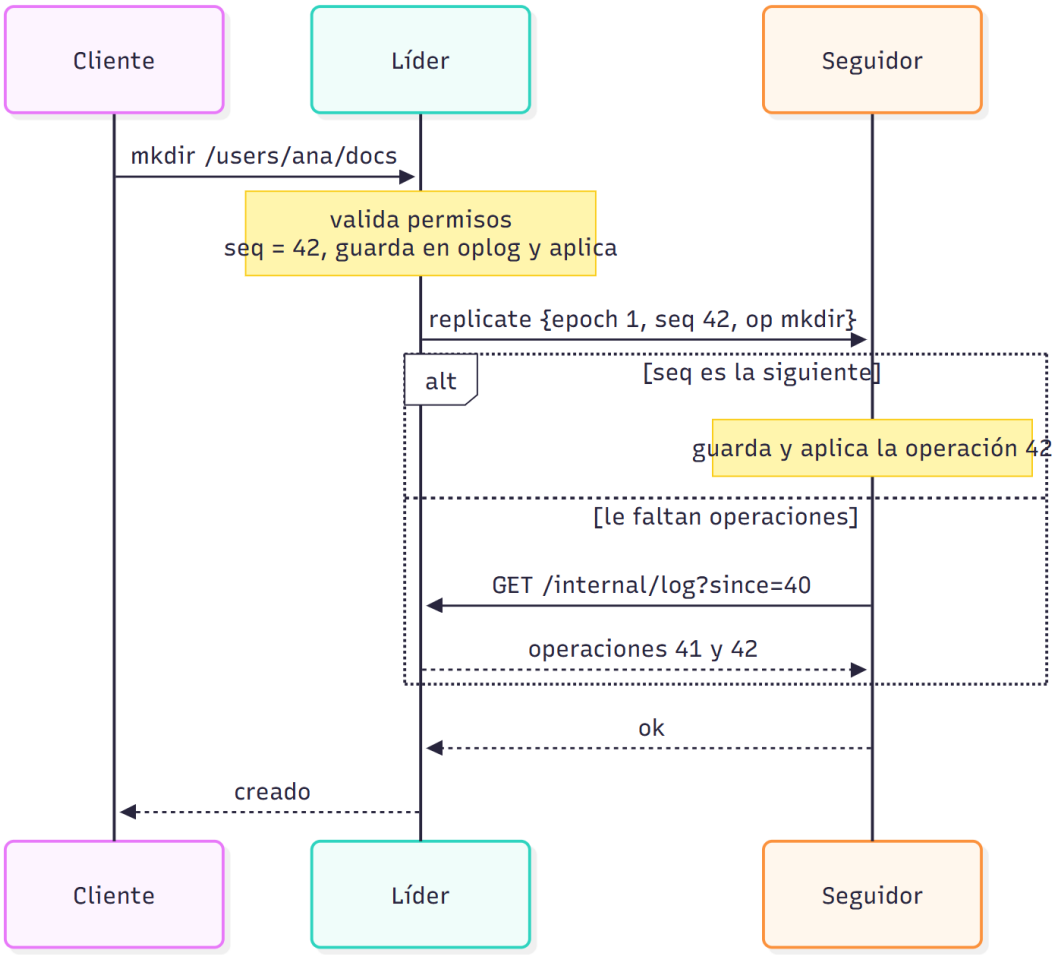


Figura 7. Replicación de los metadatos

Si el seguidor está caído, el líder no lo espera: sigue solo y el seguidor se pone al día con el log cuando vuelve.

7.3 Modelo de consistencia

Metadatos. Todas las operaciones pasan por un solo líder que les asigna un orden, y el seguidor las aplica en ese mismo orden: consistencia secuencial. Además el cliente lee los metadatos del líder, así que siempre ve lo último confirmado.

Archivos. Modelo WORM con tres reglas:

1. **Commit atómico.** Un archivo solo aparece en `ls` y solo se puede leer cuando todos sus bloques se escribieron y el cliente hizo `commit`, que es una sola operación del log. Si el cliente se cae a mitad de un `put`, el archivo no existe y los bloques huérfanos se borran a los 15 minutos.
2. **Locks de lectores y escritores.** Para escribir hace falta un lock exclusivo y para leer uno compartido. Varios pueden leer a la vez, pero mientras alguien escribe nadie más lee ni escribe ese archivo.
3. **Locks con vencimiento.** Si el cliente que tiene un lock se cae, el lock vence solo y el archivo no queda bloqueado para siempre.

Cada sobrescritura sube la versión del archivo; los bloques de la versión anterior pasan a la tabla `garbage` y el líder ordena borrarlos en el siguiente heartbeat.

Se descartó multilíder porque habría que resolver conflictos, y la estrategia más simple (última escritura gana) pierde datos sin avisar. Para metadatos, que son pocos y livianos, un solo líder aguanta la carga.

7.4 DFSha frente al teorema CAP

Situación	Qué hace DFSha	Qué prioriza
Cae un DataNode	Sigue leyendo y escribiendo con las otras réplicas	Disponibilidad, sin perder consistencia porque los bloques no cambian
Cae el seguidor	El líder sigue solo y el seguidor se sincroniza al volver	Disponibilidad
Cae el líder	Durante unos 5 segundos las operaciones de metadatos esperan, luego atiende el seguidor	Consistencia, nunca hay dos líderes atendiendo
Se corta la red solo entre los dos ControlNodes	Los dos pueden creerse líderes un rato. Al reconectarse queda el de mayor epoch y el otro copia su log	Disponibilidad durante el corte, a costa de perder lo escrito en el lado que pierde

8. Alta disponibilidad

Falla	Cómo se detecta	Cómo se recupera
Cae un DataNode	10 s sin heartbeat	Las lecturas usan otra réplica de inmediato; el líder re-replica los bloques que quedaron con 2 copias
Un DataNode cae durante un `put`	El nodo anterior de la cadena no lo contacta	Salta al siguiente; con 2 copias el bloque se acepta
Un bloque está dañado	El checksum SHA-256 no coincide al leer	El cliente lee otra réplica
Cae el ControlNode líder	El seguidor no recibe respuesta 3 veces seguidas (cada 2 s)	El seguidor se vuelve líder con epoch mayor; el SDK busca al nuevo líder y reintenta
Cae el ControlNode seguidor	El líder no lo contacta	El líder sigue solo; al volver, el seguidor copia el log
Vuelve un nodo caído	Su heartbeat o su consulta de estado	El DataNode reporta sus bloques; el ControlNode vuelve como seguidor y se sincroniza

8.1 Re-replicación

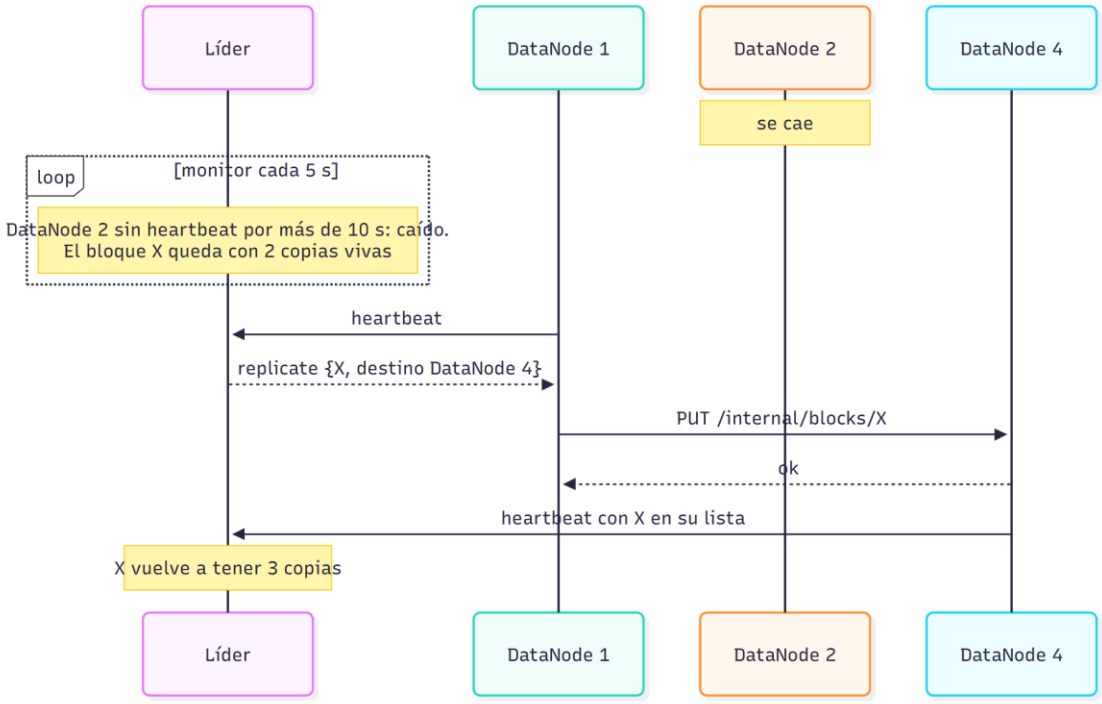


Figura 8. Re-replicación

El monitor del líder revisa todos los bloques. Si uno tiene menos copias vivas que el factor de replicación, elige como origen un nodo que lo tenga y como destino el nodo vivo con menos bloques que no lo tenga, y le manda la orden al origen en su próximo heartbeat. Espera 15 s antes de repetir la misma orden. Un líder recién elegido espera 20 s antes de re-replicar para que todos los DataNodes alcancen a reportarse.

8.2 Cambio de líder

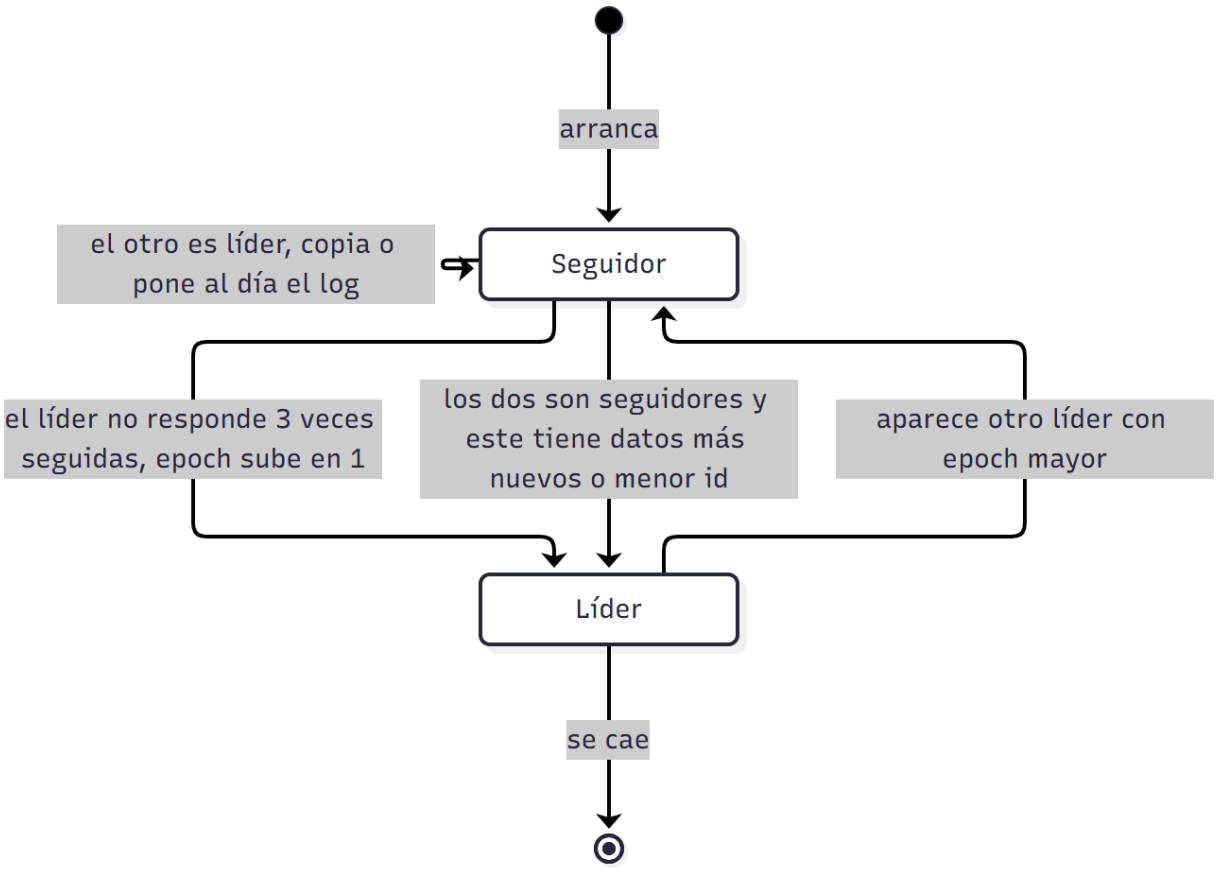


Figura 9. Cambio de líder

Cada ControlNode le pregunta al otro su estado cada 2 segundos. El **epoch** es un número que sube con cada líder nuevo: si un líder viejo intenta replicar, el seguidor lo rechaza porque tiene un epoch mayor y el viejo deja de ser líder. Si los dos arrancan juntos, lidera el del log más largo y, si empatan, el de menor id.

Los DataNodes mandan heartbeat a los dos ControlNodes, así el seguidor ya tiene el mapa de bloques cuando se vuelve líder. El cliente conoce las dos direcciones; si el líder no responde, el SDK pregunta el estado de cada una hasta encontrar al nuevo y repite la operación.

Los locks y las subidas sin commit viven en la memoria del líder y se pierden en un cambio de líder. Como los locks vencen y el cliente reintenta, no queda nada inconsistente: la operación en curso falla y se repite.

9. Escalabilidad y rendimiento

9.1 Decisiones

1. **Separar control y datos.** El ControlNode solo maneja metadatos y nunca mueve contenido de archivos; los bloques van directo entre cliente y DataNodes.
2. **DataNodes horizontales.** Para sumar capacidad se cambia `datanode_count` en Terraform. El nodo nuevo se registra con su primer heartbeat y el round robin empieza a usarlo.
3. **Paralelismo.** El cliente sube y baja varios bloques a la vez (4 hilos por defecto).
4. **Lecturas repartidas** entre las tres réplicas, en orden aleatorio.
5. **Lectura parcial.** `open` y `read` bajan solo los bloques que cubren el rango pedido.
6. **Copia en cadena.** El cliente sube cada bloque una sola vez; las otras dos copias salen de los DataNodes.

9.2 Límites

Dimensión	Hasta dónde llega	Qué la limita
Tamaño de archivo	Sin tope práctico; el cliente trabaja por bloques sin cargar todo en memoria	Disco de los DataNodes
Cantidad de archivos	Cientos de miles de filas en SQLite	Memoria y disco del ControlNode, como el NameNode de HDFS
DataNodes	Decenas	Tamaño de los reportes de bloques en cada heartbeat
Operaciones por segundo	Ver sección 13.9	Un solo líder serializa las escrituras de metadatos

9.3 Ley de Little

Con $L = \lambda \cdot W$ se calcula cuántas operaciones hay dentro del sistema a la vez. Los valores medidos están en la sección 13.8.

10. Seguridad

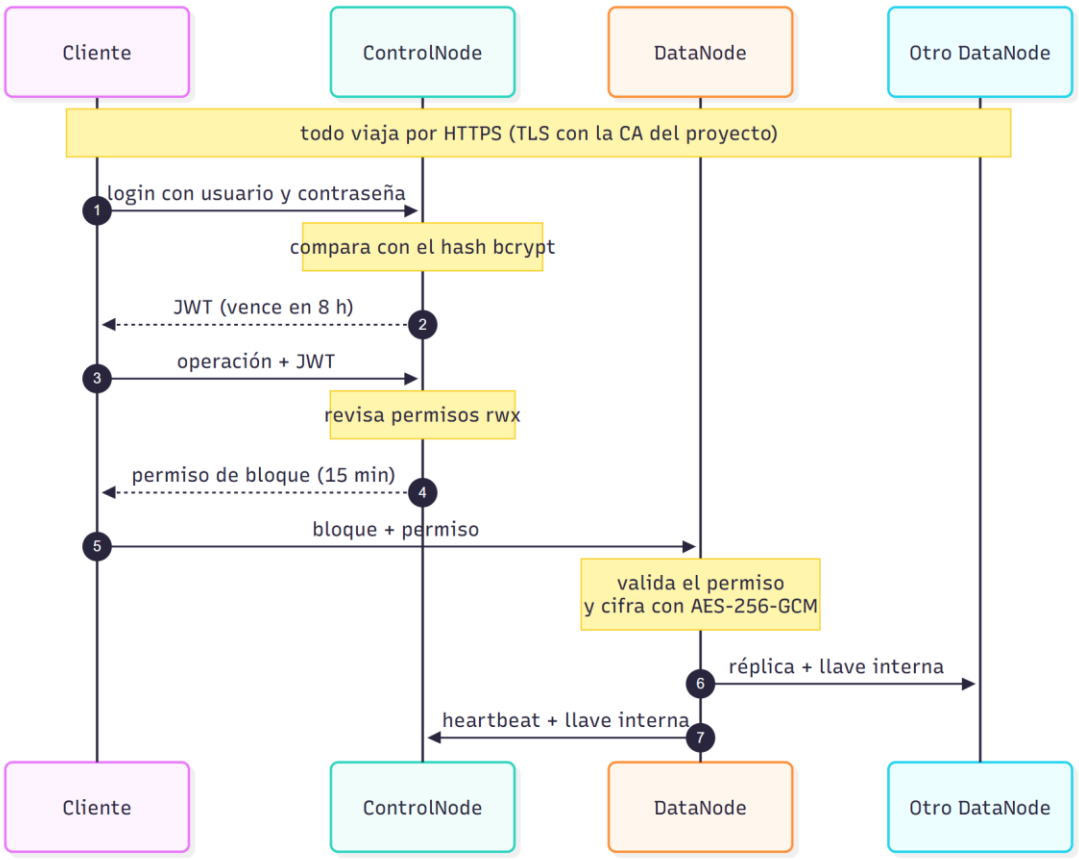


Figura 10. Seguridad

Requisito	Mecanismo	Detalle
Cifrado en tránsito	TLS en todos los nodos	Una CA propia firma un certificado por nodo. El cliente y los nodos solo aceptan servidores firmados por esa CA. Sin TLS no hay conexión
Cifrado en reposo	AES-256-GCM por bloque	Cada DataNode tiene su propia llave y el id del bloque va como dato asociado, así un bloque no se puede hacer pasar por otro. GCM también detecta si el archivo en disco fue alterado. Los discos EBS van cifrados
Usuarios	bcrypt y JWT	Las contraseñas se guardan con hash y sal. El login devuelve un JWT que vence en 8 horas
Control de acceso	Permisos tipo Linux	Cada archivo y carpeta tiene dueño, grupo y permisos `rwx`. Leer pide `r`, crear o borrar pide `w` en la carpeta padre. Lo nuevo hereda el grupo de su carpeta. Solo el dueño cambia permisos y solo el admin crea usuarios
Acceso a bloques	Permiso de bloque	El DataNode no conoce usuarios: solo acepta una lectura o escritura con un token firmado por el ControlNode para ese bloque y esa operación, válido 15 minutos

Requisito	Mecanismo	Detalle
Entre nodos	Llave interna y red privada	Los endpoints `/internal/*` exigen una llave compartida y que la petición venga de la VPC. Desde Internet se rechazan
Red	Security Group	Abiertos a Internet solo el 8000 y el 8100 para el cliente y el 22 para SSH, que se puede limitar a una IP
Secretos	Fuera del código	Se generan al azar y se pasan por variables de entorno. `.env`, `certs/` y `terraform.tfvars` están en `.gitignore`

11. Transparencia

1. **De acceso.** El usuario usa comandos como los de Linux (`ls`, `cd`, `mkdir`, `rm`, `cat`) y rutas como `/users/ana/tesis.pdf`, sin importar que el archivo esté partido en 50 bloques en 4 servidores. Esas rutas son entradas del catálogo, no carpetas reales de ningún disco.
2. **De ubicación.** El cliente nunca guarda direcciones de DataNodes. El ControlNode le dice dónde están los bloques en cada operación, así que si un bloque se re-replica a otro nodo, la siguiente lectura ya va al nuevo.
3. **De fallas.** Si un DataNode no responde, el SDK lee de otra réplica; si cae el líder, encuentra al nuevo.
4. **De replicación.** El usuario ve un solo archivo aunque haya tres copias de cada bloque.

12. Entorno de ejecución

12.1 Tecnologías

Pieza	Tecnología
Lenguaje	Python 3.12
APIs	FastAPI y Uvicorn con TLS
Cliente HTTP	httpx
Metadatos	SQLite en modo WAL
Seguridad	cryptography (AES-GCM y certificados), bcrypt, PyJWT
Contenedores	Docker (una imagen para ControlNode y DataNode) y Docker Compose
Infraestructura	Terraform con los proveedores aws y random
Pruebas	pytest y Locust

12.2 Tres formas de ejecutarlo

1. **Procesos locales** (`scripts/cluster_local.py`): 2 ControlNodes y 4 DataNodes en la misma máquina. Lo usan las pruebas automáticas porque pueden apagar y prender nodos.
2. **Docker Compose** (`docker-compose.yml`): 6 contenedores en una red privada de Docker.
3. **AWS con Terraform** (`infra/terraform`): el despliegue real.

12.3 Despliegue en AWS

Terraform crea una VPC `10.0.0.0/16` con una subred `10.0.1.0/24`, un Internet Gateway, un Security Group, secretos aleatorios y una EC2 por nodo (Ubuntu 24.04, disco cifrado, IP privada fija). Al arrancar, cada EC2 ejecuta un script que instala Docker, escribe los certificados y la configuración, consulta su propia IP pública, clona el repositorio, construye la imagen y levanta el contenedor con reinicio automático.

```
python scripts/generar_seguridad.py
cd infra/terraform
terraform init
terraform apply
dfsha config <comando_config que imprime terraform>
```

Para escalar se cambia `datanode_count` (entre 3 y 6) y se aplica de nuevo. Para borrar todo, `terraform destroy`.

Recurso	Valor
Región	us-east-1 (AWS Academy Learner Lab)
Máquinas	6 EC2 `t3.small`: 2 ControlNodes y 4 DataNodes
IP privadas	ControlNodes 10.0.1.10 y 10.0.1.11; DataNodes 10.0.1.21 a 10.0.1.24
Recursos creados por `terraform apply`	19 (VPC, subred, gateway, tabla de rutas y su asociación, Security Group, 6 EC2 y 7 secretos)
Tiempo hasta que el clúster responde	Entre 3 y 5 minutos después del `apply`

Estado del clúster justo después del despliegue, con el comando `dfsha nodes`:

```
ControlNodes:
  controlnode1   leader    epoch 1  operaciones en el log: 1
  controlnode2   follower  epoch 1  operaciones en el log: 1
DataNodes (factor de replicación 3):
  datanode1    UP    0 bloques    0 B
  datanode2    UP    0 bloques    0 B
  datanode3    UP    0 bloques    0 B
  datanode4    UP    0 bloques    0 B
Archivos: 0  Bloques: 0  Bloques con pocas réplicas: 0  Bloques perdidos: 0
```

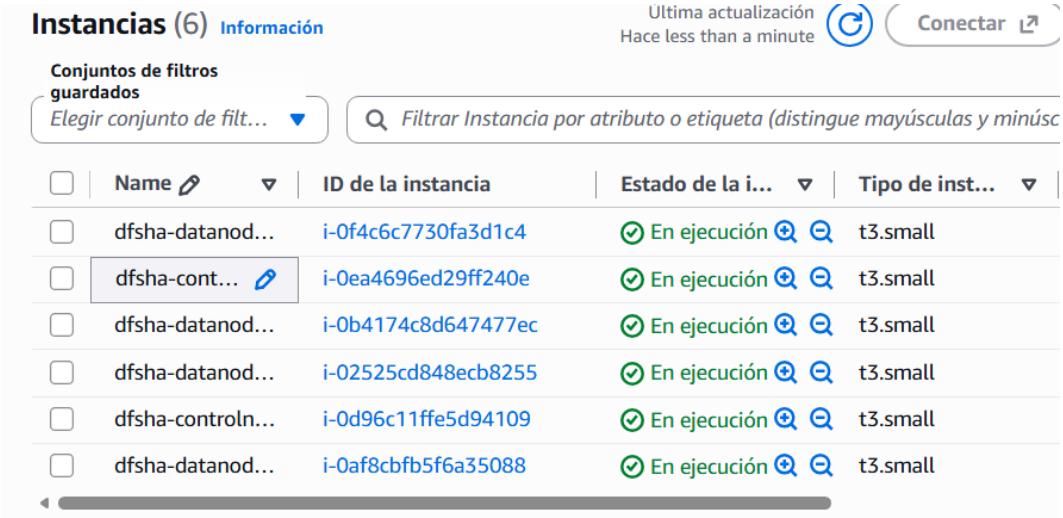


Figura 11. Las 6 instancias del clúster en la consola de EC2

13. Pruebas y resultados

13.1 Plan

Id	Prueba	Cómo se ejecutó
P1	Funcionales: gestión, transferencia, acceso y CLI	pytest, clúster local
P2	Distribución de bloques	AWS, video de 190 MB
P3	Caída de un DataNode	AWS, apagando un contenedor
P4	Caída del ControlNode líder	AWS, matando el contenedor
P5 y P6	Locks, commit atómico y versiones	pytest local y comprobación en AWS
P7	Seguridad	pytest local y comprobación en AWS
P8	Carga con usuarios simultáneos	Locust desde una EC2 dentro de la VPC
P9	Escalabilidad con 3, 4 y 5 DataNodes	Locust y Terraform
P10	Transferencia según tamaño y número de hilos	`scripts/benchmark.py`, desde mi casa y desde la VPC

Los archivos de prueba son un libro en PDF de 42,7 MB que tenía de un trabajo y un video de 190,1 MB que tenía en mi carpeta, además de bytes aleatorios generados por las pruebas. La integridad se comprueba comparando el SHA-256 del archivo original con el del descargado.

13.2 Pruebas automáticas (P1 a P7)

Levantan un clúster local de 2 ControlNodes y 4 DataNodes con bloques de 1 MB y heartbeats cortos.

Prueba	Casos	Resultado
P1 Funcionales	12	12 de 12
P2 Particionamiento	2	2 de 2
P3 y P4 Fallas	3	3 de 3
P5 y P6 Consistencia	7	7 de 7
P7 Seguridad	9	9 de 9
Total	**33**	**33 de 33, en menos de dos minutos**

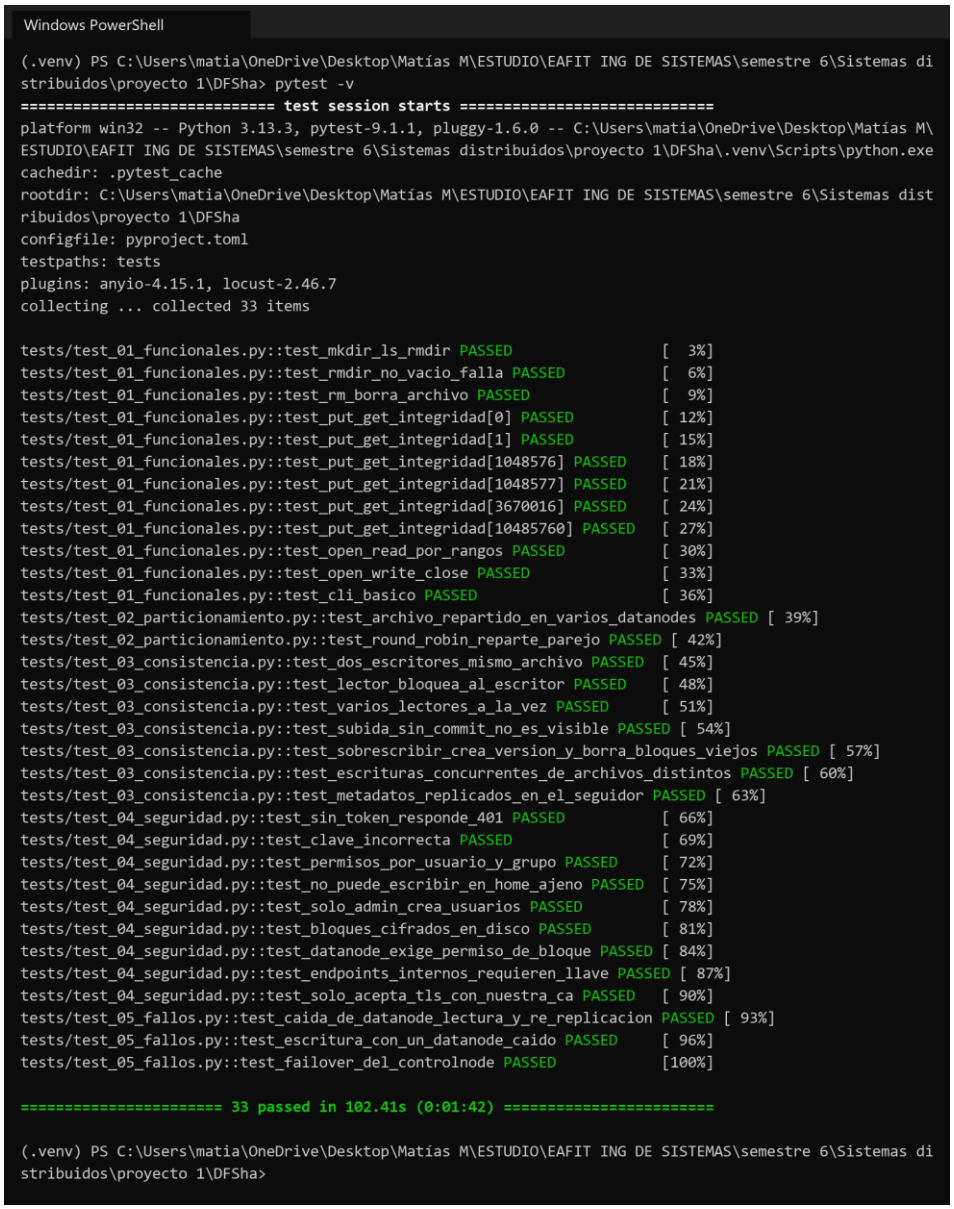


Figura 12. Salida de pytest -v: las 33 pruebas pasan

13.3 P2: distribución de bloques en AWS

Se subió `vide190mb.mp4` (190,1 MB) como usuario ana: 48 bloques, 144 copias entre los 4 DataNodes.

DataNode	Copias	Porcentaje
datanode1	36	25 %
datanode2	36	25 %
datanode3	36	25 %
datanode4	36	25 %

Cada nodo guarda exactamente la cuarta parte y ninguno tiene el archivo completo. Al bajarlo, los 48 checksums coincidieron y el SHA-256 del archivo descargado fue igual al del original. Desde mi casa, la subida tardó 41,6 s (4,6 MB/s) y la descarga 66,8 s (2,8 MB/s).

Salida de `dfsha info libro.pdf` (42,7 MB, 11 bloques):

```
/users/ana/libro.pdf  42.7 MB  versión 1  11 bloques de hasta 4.0 MB
  bloque 0   a34b51053218   4.0 MB  -> datanode1, datanode2, datanode3
  bloque 1   191768809ad2   4.0 MB  -> datanode2, datanode3, datanode4
  bloque 2   b8770d38b42e   4.0 MB  -> datanode1, datanode3, datanode4
  bloque 3   db4336698c86   4.0 MB  -> datanode1, datanode2, datanode4
  ...
  bloque 10  e1553469fa54   2.7 MB  -> datanode1, datanode3, datanode4
```

13.4 P3: caída de un DataNode en AWS

Se subió una copia nueva del libro (11 bloques, 9 con una copia en datanode2) y se apagó datanode2 con `docker stop`. Los tiempos se miden desde el momento en que el comando termina.

Medición	Resultado
Descarga del libro con los 4 nodos	13,0 s
Descarga justo después de apagar datanode2	14,3 s en una corrida y 19,9 s en otra, con el mismo SHA-256 que el original
El ControlNode marca datanode2 como DOWN	9,0 s
Se ven bloques con pocas réplicas	12,1 s
Todos los bloques otra vez con 3 copias vivas	16,8 s

Antes de apagarlo, los bloques 0 a 3 estaban repartidos entre los 4 nodos. Después, los cuatro estaban en datanode1, datanode3 y datanode4.

La lectura no se interrumpió. El cliente prueba una réplica y, si no responde, pasa a la siguiente, por eso la descarga tardó entre 1 y 7 segundos más. La caída se detectó a los 9 s (el límite es 10 s sin heartbeat) y antes de 17 s cada bloque volvió a tener 3 copias.

13.5 P4: caída del ControlNode líder en AWS

Se mató el contenedor de controlnode1 (líder) con `docker kill` para simular una falla abrupta.

Medición	Resultado
controlnode2 se vuelve líder	5,0 s después, con epoch 2
Primer `ls` exitoso desde el mismo cliente	5,3 s, mostrando todos los archivos
`put` de 10 MB con el seguidor como líder	3,4 s, y el `get` devolvió el mismo contenido
controlnode1 vuelve a arrancar	Queda como seguidor y se sincroniza en 1,7 s
Operaciones en el log al final	6 en los dos nodos, incluido el archivo escrito durante la falla

El cliente no tuvo que reconfigurarse: el SDK preguntó cuál nodo era líder y repitió la operación. El usuario solo notó una pausa de unos 5 segundos.

13.6 P5 y P6: consistencia en AWS

Caso	Esperado	Obtenido
Luis escribe un archivo mientras Ana tiene un lock de escritura	Error 409	409
Luis escribe después de que Ana suelta el lock	Permitido	Permitido
Subida interrumpida antes del commit	El archivo no aparece	No aparece en `ls`
Sobrescribir un archivo (en pytest)	Versión 2 y bloques viejos borrados	Correcto

13.7 P7: seguridad en AWS

Caso	Esperado	Obtenido
Petición sin token	401	401
Clave incorrecta	401	401
Pedro lee un archivo privado de Ana	403	403
Pedro escribe en la carpeta de Ana	403	403
Pedro intenta crear usuarios	403	403
Luis, del mismo grupo, lee el libro compartido	Permitido	Permitido
`/internal/heartbeat` sin llave	403	403
`/internal/log` con llave falsa	403	403
DataNode: bloque sin permiso	401	401
DataNode: bloque con permiso	200	200
Cliente sin la CA del proyecto	Rechazado	Rechazado (error de certificado)

Caso	Esperado	Obtenido
HTTP sin cifrar	Rechazado	Rechazado

También se revisó un bloque en el disco de un DataNode: sus primeros bytes fueron `cc 09 a5 f4 69 ea 14 d4 70 c1 82 58 ...`, sin ningún texto legible.

13.8 P8: carga con Locust

La carga se generó desde una EC2 `t3.small` temporal dentro de la misma VPC, porque desde mi casa la conexión (cerca de 4 MB/s de subida) limitaba todo lo que se medía. Cada usuario simulado hace login y repite, con una espera de 0,5 a 2 s entre operaciones: subir un archivo de 64 KB, 512 KB o 2 MB (20 % de las operaciones), descargar uno propio (50 %) y listar su carpeta (30 %). Cada corrida duró 2 minutos con 4 DataNodes.

Usuarios	Operaciones	Errores	Operaciones/s	Mediana (ms)	p95 (ms)	Media (ms)	L = λ · W
10	893	0	7,5	26	100	35	0,3
50	4.243	0	35,6	37	140	60	2,1
100	7.047	0	59,3	95	760	226	13,4

Con 50 usuarios, por tipo de operación: listar tardó 7 ms de mediana (p95 29), descargar 42 ms (p95 100), y subir 53 ms (64 KB), 72 ms (512 KB) y 120 ms (2 MB).

No hubo ningún error en las tres corridas. Con 100 usuarios, λ = 59,3 operaciones por segundo y W = 0,226 s dan L = 13,4: había en promedio 13 operaciones dentro del sistema y el resto de los usuarios estaba en su tiempo de espera. La latencia sube con la carga (la mediana pasa de 26 a 95 ms y el p95 de 100 a 760 ms) pero el sistema siguió respondiendo todo.

13.9 P9: escalabilidad

Misma prueba de 100 usuarios, cambiando los DataNodes con `terraform apply`.

DataNodes	Operaciones/s	Mediana (ms)	p95 (ms)	Errores	Bloques que recibe cada nodo
3	62,1	87	370	0	100 %
4	59,3	95	760	0	75 %
5	61,5	85	360	0	60 %

La última columna sale de que cada bloque tiene 3 copias repartidas entre N nodos, así cada nodo recibe 3/N de las escrituras. Se comprobó en la corrida con 5 nodos: se agregaron 2 DataNodes vacíos y al terminar tenían unos 900 bloques cada uno, la misma cantidad que sumaron los tres nodos anteriores durante la prueba.

El caudal en operaciones por segundo no cambió con el número de nodos, y la explicación no es que el sistema no escale. Con espera de 0,5 a 2 s entre operaciones, 100 usuarios no pueden pasar de unas 70 a 80 operaciones por segundo, y ningún componente estaba cerca de su límite. Con 5 nodos, a mitad de la corrida, el uso de CPU fue: líder 42 % de un núcleo, DataNodes entre 15 y 19 % y máquina de carga 44 %. Para forzar un límite se repitió con 5 nodos sin espera entre operaciones: dio 49 operaciones por segundo con mediana de 1,4 s, y ahí la máquina de carga usaba casi un núcleo completo mientras el líder estaba al 4 % y los DataNodes entre 2 y 15 %. El límite lo ponía el generador de carga, no el clúster, así que con este montaje no se puede medir cuánto aguanta el sistema. Lo que sí se verificó es que cada nodo que se agrega se registra solo, empieza a recibir bloques y reduce la carga de los demás.

El p95 de 4 nodos fue más alto que el de 3 y el de 5. Fue la primera corrida de 100 usuarios con las máquinas recién creadas y no se repitió, así que no se saca ninguna conclusión de esa diferencia.

13.10 P10: transferencia según tamaño y número de hilos

Mediana de 3 repeticiones, con archivos de bytes aleatorios. "Casa" es mi PC y "VPC" es la EC2 de carga dentro de AWS.

Tamaño	Hilos	Casa: put (MB/s)	Casa: get (MB/s)	VPC: put (MB/s)	VPC: get (MB/s)
1 MB	1	0,65	0,91	13,4	41,0
1 MB	4	0,82	0,78	14,5	34,5
10 MB	1	2,15	2,23	29,0	76,4
10 MB	4	3,04	1,84	49,7	92,6
50 MB	1	3,34	2,69	30,8	89,3
50 MB	4	3,75	3,03	69,5	87,5
100 MB	1	no medido	no medido	30,8	80,3
100 MB	4	no medido	no medido	77,6	128,2

Desde mi casa el límite es la conexión, y el número de hilos casi no cambia nada. Dentro de AWS, con archivos grandes, usar 4 hilos en vez de 1 multiplica por 2,5 la velocidad de subida (de 30,8 a 77,6 MB/s con 100 MB) y por 1,6 la de bajada. Con archivos de 1 MB (un solo bloque) los hilos no ayudan porque no hay bloques que paralelizar.

13.11 Observaciones durante las pruebas

1. **Nodos nuevos vacíos.** Al agregar datanode4 y datanode5 empezaron con 0 bloques mientras los otros tenían más de 4.000, porque el round robin solo decide dónde van los bloques nuevos.
2. **`docker stop` puede tardar 10 s** en cerrar un DataNode; por eso en P4 se usó `docker kill`.
3. **Un DataNode dado de baja** sigue apareciendo como DOWN en `dfsha nodes` hasta que se reinicien los ControlNodes.

4. **Copias de más.** Cuando un DataNode que estuvo caído vuelve, sus bloques que ya se habían re-replicado quedan con 4 copias y las sobrantes no se borran.

14. Limitaciones y trabajo futuro

1. **Rebalanceo.** Un nodo nuevo solo recibe bloques nuevos. Falta un proceso que mueva bloques viejos hacia él, como el balanceador de HDFS.
2. **El log crece sin límite.** Se podría tomar un snapshot periódico y cortar el log.
3. **Un solo líder de metadatos** limita las operaciones por segundo; escalarlo implicaría repartir el árbol de directorios entre varios ControlNodes.

15. Conclusiones

1. Separar los metadatos de los datos funcionó: con 60 operaciones por segundo el líder usó el 42 % de un núcleo, y con 4 hilos dentro de AWS la subida de archivos grandes pasó de 31 a 78 MB/s.
2. Con tres copias por bloque, la caída de un DataNode no interrumpió las lecturas y en menos de 17 segundos todos los bloques volvieron a tener tres copias.
3. El cambio de líder tardó 5 segundos y no se perdió ningún dato. Su límite es que, con solo dos ControlNodes, un corte de red entre ellos puede dejar dos líderes un rato.
4. El modelo WORM con commit atómico y locks evitó escrituras simultáneas y archivos a medio escribir, y las 12 comprobaciones de seguridad en AWS dieron el resultado esperado.
5. La capacidad crece sumando DataNodes, que se registran solos y reducen la carga por nodo. Falta el rebalanceo de los bloques viejos, y el líder de metadatos es el límite de diseño para las operaciones por segundo.
6. La medición del rendimiento depende de desde dónde se mide: desde mi casa se midió la conexión a Internet, y solo con la carga generada dentro de la VPC se vio el comportamiento del sistema.